COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 17, 2026	**DATE SUBMITTED:**	July 21, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Hands-on Activity : Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

Screenshot

main.cpp > Student

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
    /* CONSTRUCTOR */
    Student(std::string newName = "Eydis", int newAge = 18){

        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "CALLED CONSTRUCTOR" << std::endl;

    };
    /* DECONSTRUCTOR */
    ~Student(){

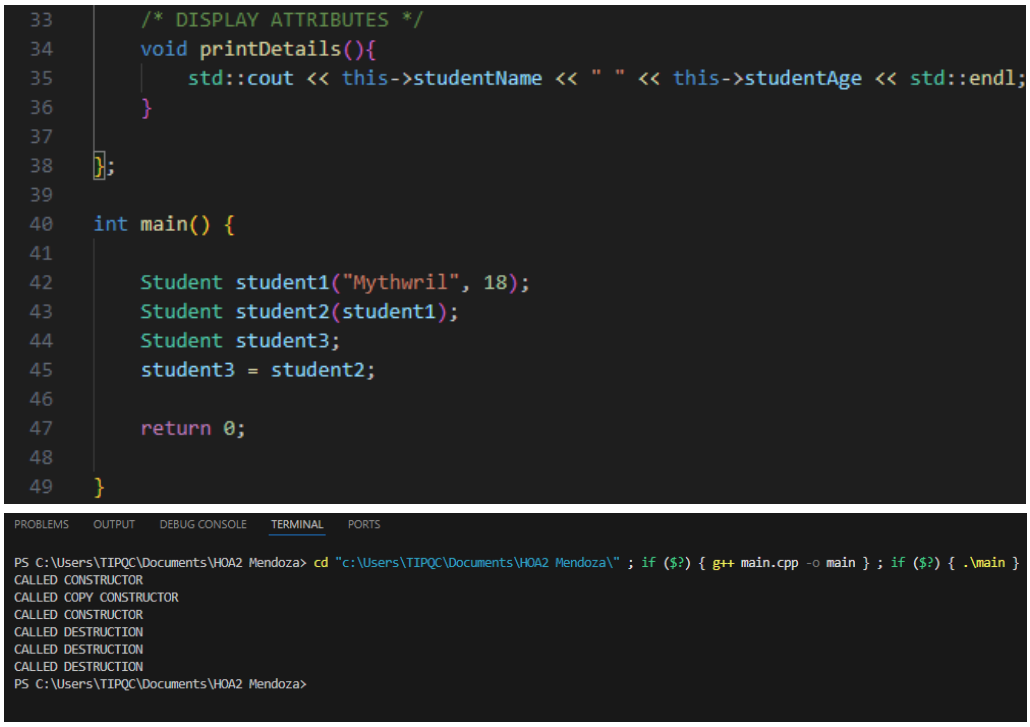
        std::cout << "CALLED DESTRUCTION" << std::endl;

    }

    /*  COPY CONSTRUCTOR */
    Student(const Student &copyStudent){

        std::cout << "CALLED COPY CONSTRUCTOR" << std::endl;
        studentName = copyStudent.studentName;
        studentAge =  copyStudent.studentAge;
    }

```



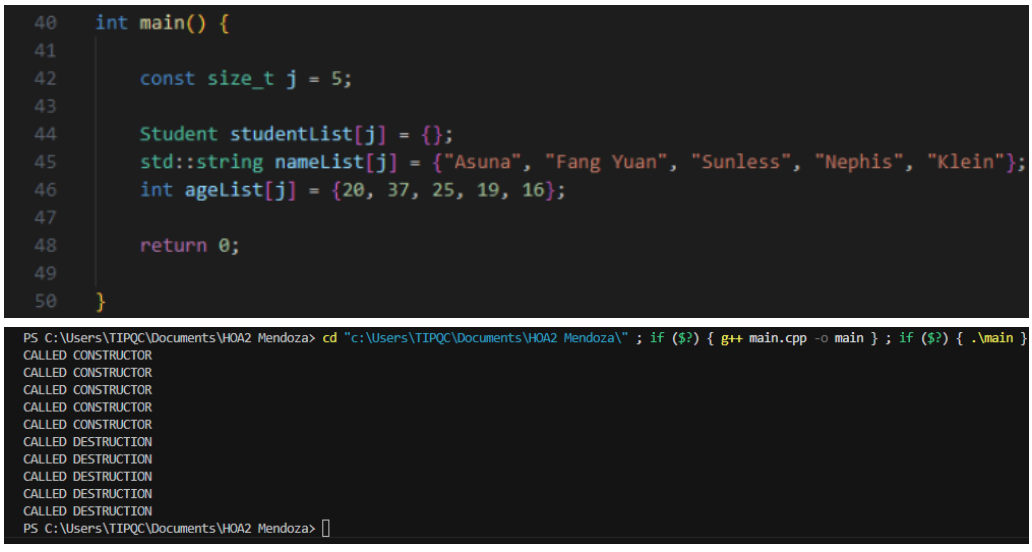
```
33      /* DISPLAY ATTRIBUTES */
34      void printDetails(){
35          std::cout << this->studentName << " " << this->studentAge << std::endl;
36      }
37
38  };
39
40  int main() {
41
42      Student student1("Mythwril", 18);
43      Student student2(student1);
44      Student student3;
45      student3 = student2;
46
47      return 0;
48
49  }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\Documents\HOA2 Mendoza> cd "c:\Users\TIPQC\Documents\HOA2 Mendoza\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
CALLED CONSTRUCTOR
CALLED COPY CONSTRUCTOR
CALLED CONSTRUCTOR
CALLED DESTRUCTION
CALLED DESTRUCTION
CALLED DESTRUCTION
PS C:\Users\TIPQC\Documents\HOA2 Mendoza>
```

Observation	The program first reads which student with its respective variables to be specific the the Student name and age. Then, it refers to the public class of Student to construct and reads the main program of student 1 and prints its respective setted name and age. After that prints calls to the constructor after its function to finish. Then it goes on a loop where it does the same thing via copying its constructor.

Table 2.1. Initial Driver Program

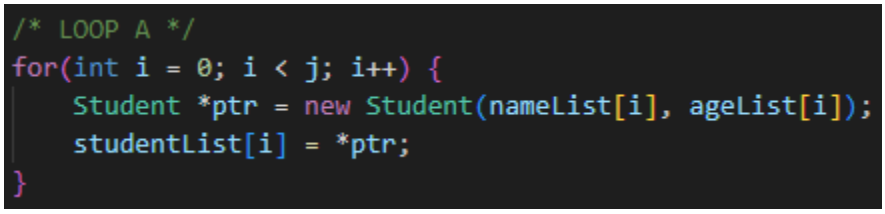
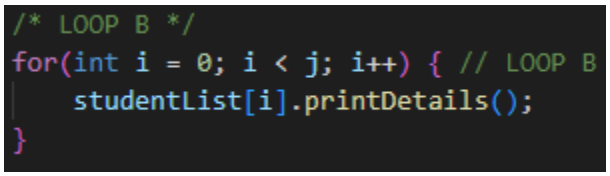
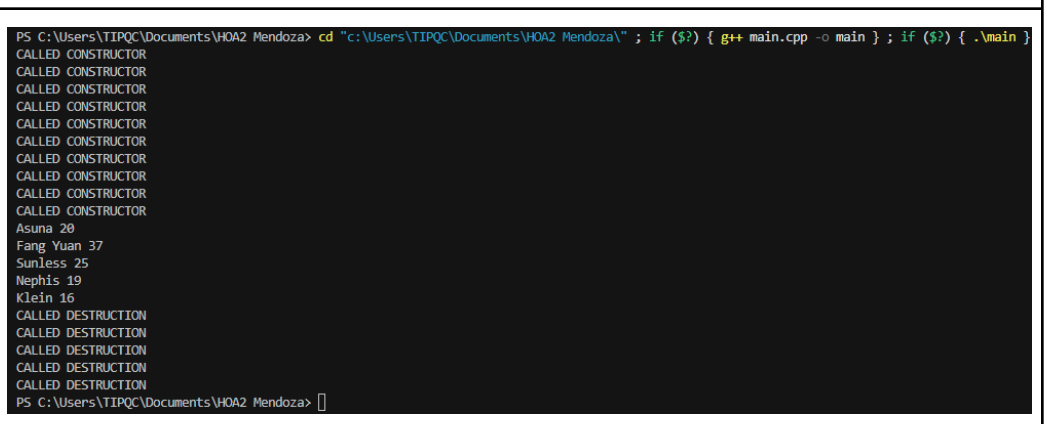
Screenshot	

```
40  int main() {
41
42      const size_t j = 5;
43
44      Student studentList[j] = {};
45      std::string nameList[j] = {"Asuna", "Fang Yuan", "Sunless", "Nephis", "Klein"};
46      int ageList[j] = {20, 37, 25, 19, 16};
47
48      return 0;
49
50  }
```

```
PS C:\Users\TIPQC\Documents\HOA2 Mendoza> cd "c:\Users\TIPQC\Documents\HOA2 Mendoza\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
CALLED CONSTRUCTOR
CALLED CONSTRUCTOR
CALLED CONSTRUCTOR
CALLED CONSTRUCTOR
CALLED CONSTRUCTOR
CALLED DESTRUCTION
CALLED DESTRUCTION
CALLED DESTRUCTION
CALLED DESTRUCTION
CALLED DESTRUCTION
PS C:\Users\TIPQC\Documents\HOA2 Mendoza>
```

Observation	The program sets the studentlist which an array to none or blank then next it sets a limit by reading the strings inside the array and sets it to 5. And then goes to the constructor function and places its respective variable which was created on construction, basically setting a temporary name and age for the student list that loops 5 times. Then after that, destroy each and one of the student names of strings and age 5 times. But it does not necessarily print their respective name and age that was created on the main program.

Table 2.2. Modified Driver Program with Student Lists

Loop A	
Observation	This refers to the main program by creating an address for Loop B to refer to and then swapping or updating it to the respective student names and age on the array created on the main program. Basically, replacing the temporary names and age on the construct function.
Loop B	
Observation	This means that its printing its details via addresses by referring to the addresses Loop A has created for it to print the respective details.
Output	
Observation	The output represents that it calls the constructor first so that it can refer to student name and age thats why it printed 10 times since it has 5 student names and 5 student ages as basically it created a temporary student name and age before it gets replaced by the main programs student list or an array with its respective student name and age and refers by it and applies a loop to print the newly created student list which this represents the replacement of the

	temporary one created by the construction function. Then after that, destroy each and one of the student names of strings and age 5 times. The reason why it didn't call destruction 10 times and only 5 is because the student name and age is part of each other making it 5.

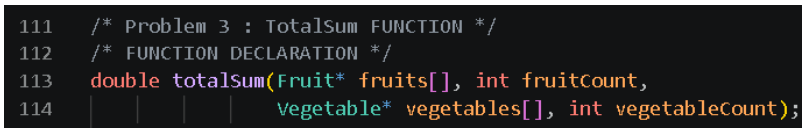
Table 2.3. Final Driver Program

Modifications	```
	int main() {
	
	    const size_t j = 5;
	
	    Student studentList[j] = {};
	    std::string nameList[j] = {"Asuna", "Fang Yuan", "Sunless", "Nephis", "Klein"};
	    int ageList[j] = {20, 37, 25, 19, 16};
	
	    /* LOOP A */
	    for(int i = 0; i < j; i++) {
	        Student *ptr = new Student(nameList[i], ageList[i]);
	        studentList[i] = *ptr;
	
	        /* MODIFICATION */
	        delete ptr;
	        ptr = nullptr;
	    }
	
	    /* LOOP B */
	    for(int i = 0; i < j; i++) { // LOOP B
	        studentList[i].printDetails();
	    }
	
	    return 0;
	}
	``

Observation	The copy assignment operator is used when an already existing object is assigned the values of another existing object. It checks whether the object is being assigned to itself and then copies the necessary data members. This allows the existing object's values to be replaced with the values of another object. In this activity, the copy assignment operator is important because it demonstrates how an existing object can receive data from another object without creating a new object.

Table 2.4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Sections	Answer
Header File Declaration Section	
Observation	Basically iostream allows input and output function, strings for storing name for vegetable and fruit then iomanip for table designing to be specific setting precisions on how long will be the spaces will be.
Global Declaration Section	
Observation	These are the global declarations of a function; they are declared before the main program. It is here so it can be recognized when called specifically the totalsum function where it will get the sum of the total cost of Jenna's receipt.

Class Declaration and Method Definition Section	

```cpp
/* FRUIT CLASS */
class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    // CONSTRUCTOR
    Fruit(std::string n = " ", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    // DESTRUCTOR
    ~Fruit() {
        std::cout << "Fruit Destructor Called: " << name << std::endl;
    }

    // COPY CONSTRUCTOR
    Fruit(const Fruit& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    // COPY ASSIGNMENT
    Fruit& operator=(const Fruit& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    double totalSum() const {
        return price * quantity;
    }

    void receipt() const {
        std::cout << std::left
                  << std::setw(15) << "Fruit"
                  << std::setw(15) << name
                  << std::setw(12) << price
                  << std::setw(12) << quantity
                  << std::setw(12) << totalSum()
                  << std::endl;
    }
};
```

```cpp
/* VEGETABLE CLASS */
class Vegetable {
private:
    std::string name;
    double price;
    int quantity;

public:
    // CONSTRUCTOR
    Vegetable(std::string n = " ", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    // DESTRUCTOR
    ~Vegetable() {
        std::cout << "Vegetable Destructor Called: " << name << std::endl;
    }

    // COPY CONSTRUCTOR
    Vegetable(const Vegetable& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    // COPY ASSIGNMENT
    Vegetable& operator=(const Vegetable& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    double totalSum() const {
        return price * quantity;
    }

    void receipt() const {
      std::cout << std::left
                << std::setw(15) << "Vegetables"
                << std::setw(15) << name
                << std::setw(12) << price
                << std::setw(12) << quantity
                << std::setw(12) << totalSum()
                << std::endl;
    }
};
```

Observation	For this, the vegetable and fruit class has the same property; the only difference between these two is that each class has their own private data members. Which therefore, can only be accessed through class function. Additionally, this has a construct which creates an object from a certain class. Destructor for removing memory to prevent leaks. Copy Constructor to copy existing objects created by Constructors which it will allow to have its same data. Lastly, the copy assignment operator so that it prevents the object from assigning itself to itself. .The method for calculation and applied as constant to only read the object and not modify it. Receipt on the other hand for displaying information also marked as constant so it will not modify the object.

Main Function

```cpp
/* THE MAIN PROGRAM */
int main() {

    /* Problem 2: Creating a GroceryList Array */

    Fruit* fruitsTaken[2];
    Vegetable* vegetablesTaken[2];

    fruitsTaken[0] = new Fruit("Apple", 10, 7);
    fruitsTaken[1] = new Fruit("Banana", 10, 8);

    vegetablesTaken[0] = new Vegetable("Broccoli", 60, 12);
    vegetablesTaken[1] = new Vegetable("Lettuce", 50, 10);

    std::cout << "\n================ MEMORY STATUS ================\n";

    std::cout << "Apple Pointer      : " << fruitsTaken[0] << std::endl;

    std::cout << "Banana Pointer     : " << fruitsTaken[1] << std::endl;

    std::cout << "Broccoli Pointer   : " << vegetablesTaken[0] << std::endl;

    std::cout << "Lettuce Pointer    : " << vegetablesTaken[1] << std::endl;

    std::cout << "==============================================\n";

    std::cout << "============================================================\n";
    std::cout << "               JENNA'S GROCERY RECEIPT\n";
    std::cout << "============================================================\n";

    std::cout << std::left
              << std::setw(15) << "Type"
              << std::setw(15) << "Item"
              << std::setw(10) << "Price"
              << std::setw(10) << "Quantity"
              << std::setw(10) << "Total"
              << std::endl;
    std::cout << "------------------------------------------------------------\n";

    std::cout << "Fruits:" << std::endl;
    for (int i = 0; i < 2; i++) {
        fruitsTaken[i]->receipt();
    }

    std::cout << "Vegetables:" << std::endl;
    for (int i = 0; i < 2; i++) {
        vegetablesTaken[i]->receipt();
    }

    std::cout << "\nTotal Amount in Pesos: "
              << totalSum(fruitsTaken, 2, vegetablesTaken, 2)
              << std::endl;

    /* Problem 4 : Deleting Lettuce from Jenna Receipt */
```

```
117  int main() {
171  char answer;
172
173  std::cout << "\nProceed to Delete Lettuce? (Y/N): ";
174  std::cin >> answer;
175
176  if (answer == 'Y' || answer == 'y') {
177
178      std::cout << "\n========== POINTER LIST BEFORE DELETION ==========\n";
179
180      std::cout << "Apple Pointer    : " << fruitsTaken[0] << std::endl;
181      std::cout << "Banana Pointer   : " << fruitsTaken[1] << std::endl;
182      std::cout << "Broccoli Pointer : " << vegetablesTaken[0] << std::endl;
183      std::cout << "Lettuce Pointer  : " << vegetablesTaken[1] << std::endl;
184
185      std::cout << "================================================\n";
186
187      std::cout << "\nDeleting Lettuce...\n";
188
189      delete vegetablesTaken[1];
190
191      vegetablesTaken[1] = nullptr;
192
193      std::cout << "\nMemory successfully deallocated.\n";
194
195      std::cout << "\n========== POINTER LIST AFTER DELETION ==========\n";
196
197      std::cout << "Apple Pointer    : " << fruitsTaken[0] << std::endl;
198      std::cout << "Banana Pointer   : " << fruitsTaken[1] << std::endl;
199      std::cout << "Broccoli Pointer : " << vegetablesTaken[0] << std::endl;
200      std::cout << "Lettuce Pointer  : " << vegetablesTaken[1] << std::endl;
201
202      std::cout << "================================================\n";
203
204      if (vegetablesTaken[1] == nullptr) {
205          std::cout << "\nLettuce successfully deleted.\n";
206          std::cout << "Lettuce pointer is now nullptr.\n";
207      }
208  }
209      else {
210      std::cout << "\nDeletion cancelled.\n";
211  }
212
213      std::cout << "\nPress Enter to Continue...";
214      std::cin.ignore();
215      std::cin.get();
216
217      std::cout << "\n============================================================\n";
218      std::cout << "              UPDATED GROCERY RECEIPT\n";
219      std::cout << "============================================================\n";
220
221      std::cout << "Fruits:" << std::endl;
222      for (int i = 0; i < 2; i++) {
223          if (fruitsTaken[i] != nullptr) {
224              fruitsTaken[i]->receipt();
```



```
            fruitsTaken[i]->receipt();
        }
    }

    std::cout << "\nVegetables:" << std::endl;
    for (int i = 0; i < 2; i++) {
        if (vegetablesTaken[i] != nullptr) {
            vegetablesTaken[i]->receipt();
        }
    }

    std::cout << "\nNew Total Amount in Pesos: "
              << totalSum(fruitsTaken, 2, vegetablesTaken, 2)
              << std::endl;

// Cleaning up memory to prevent memory leak.

    std::cout << "\n========== POINTER LIST BEFORE FINAL CLEANUP ==========\n";

    std::cout << "Apple Pointer    : " << fruitsTaken[0] << std::endl;
    std::cout << "Banana Pointer   : " << fruitsTaken[1] << std::endl;
    std::cout << "Broccoli Pointer : " << vegetablesTaken[0] << std::endl;
    std::cout << "Lettuce Pointer  : " << vegetablesTaken[1] << std::endl;

    std::cout << "=====================================================\n";

    std::cout << "\nCleaning up remaining memory...\n";

// Delete Apple and Banana
    for (int i = 0; i < 2; i++) {
    delete fruitsTaken[i];
    fruitsTaken[i] = nullptr;
}

// Delete Broccoli
    delete vegetablesTaken[0];

    vegetablesTaken[0] = nullptr;

    std::cout << "\nMemory successfully cleaned up.\n";

    std::cout << "\n========== POINTER LIST AFTER FINAL CLEANUP ==========\n";

    std::cout << "Apple Pointer    : " << fruitsTaken[0] << std::endl;
    std::cout << "Banana Pointer   : " << fruitsTaken[1] << std::endl;
    std::cout << "Broccoli Pointer : " << vegetablesTaken[0] << std::endl;
    std::cout << "Lettuce Pointer  : " << vegetablesTaken[1] << std::endl;

    std::cout << "=====================================================\n";

    if (fruitsTaken[0] == nullptr &&
        fruitsTaken[1] == nullptr &&
        vegetablesTaken[0] == nullptr &&
        vegetablesTaken[1] == nullptr) {

    std::cout << "\nAll dynamically allocated memory has been deallocated successfully.\n";
    std::cout << "No memory leaks remain.\n";
}

    return 0;
}
```

Observation	The main function creates two arrays that store pointers to Fruit and Vegetable objects. The arrays themselves store memory addresses rather than the complete objects. This allows the program to dynamically allocate the actual grocery objects using the new operator. Displays their information and total prices, and allows the Lettuce object to be deleted. After deletion, the program updates the receipt and recalculates the total

	while skipping the nullptr pointer. Finally, the remaining dynamically allocated objects are deleted and their pointers are set to nullptr to properly clean up the memory and prevent memory leaks.
Method Definition	
Observation	The defined function totalSum is used to loop through vegetables and fruits pointer arrays and check each pointer using != nullptr before it accesses objects. Then if it's valid it calls the object's function the totalSum and adds the result to the overall total.
Code Output	

```
286   /* Problem 3 : TotalSum FUNCTION */
287   /* FUNCTION DEFINITION */
288   double totalSum(Fruit* fruits[], int fruitCount,
289                   Vegetable* vegetables[], int vegetableCount) {
290
291       double total = 0;
292
293       for (int i = 0; i < fruitCount; i++) {
294           if (fruits[i] != nullptr) {
295               total += fruits[i]->totalSum();
296           }
297       }
298
299       for (int i = 0; i < vegetableCount; i++) {
300           if (vegetables[i] != nullptr) {
301               total += vegetables[i]->totalSum();
302           }
303       }
304
305       return total;
306   }
```

```
PS C:\Users\mendo\OneDrive\Documents\C++\HoA2Mendoza\output> & .\'SupplementaryHoa2.exe'

================ MEMORY STATUS ================
Apple Pointer      : 0x1edcc3e73f0
Banana Pointer     : 0x1edcc3e7170
Broccoli Pointer   : 0x1edcc3e7930
Lettuce Pointer    : 0x1edcc3e75f0
===============================================
============================================================
              JENNA'S GROCERY RECEIPT
============================================================
Type          Item          Price     Quantity  Total
------------------------------------------------------------
Fruits:
Fruit         Apple         10        7         70
Fruit         Banana        10        8         80
Vegetables:
Vegetables    Broccoli      60        12        720
Vegetables    Lettuce       50        10        500

Total Amount in Pesos: 1370

Proceed to Delete Lettuce? (Y/N):
```

```
Proceed to Delete Lettuce? (Y/N): Y

========== POINTER LIST BEFORE DELETION ==========
Apple Pointer    : 0x1edcc3e73f0
Banana Pointer   : 0x1edcc3e7170
Broccoli Pointer : 0x1edcc3e7930
Lettuce Pointer  : 0x1edcc3e75f0
================================================

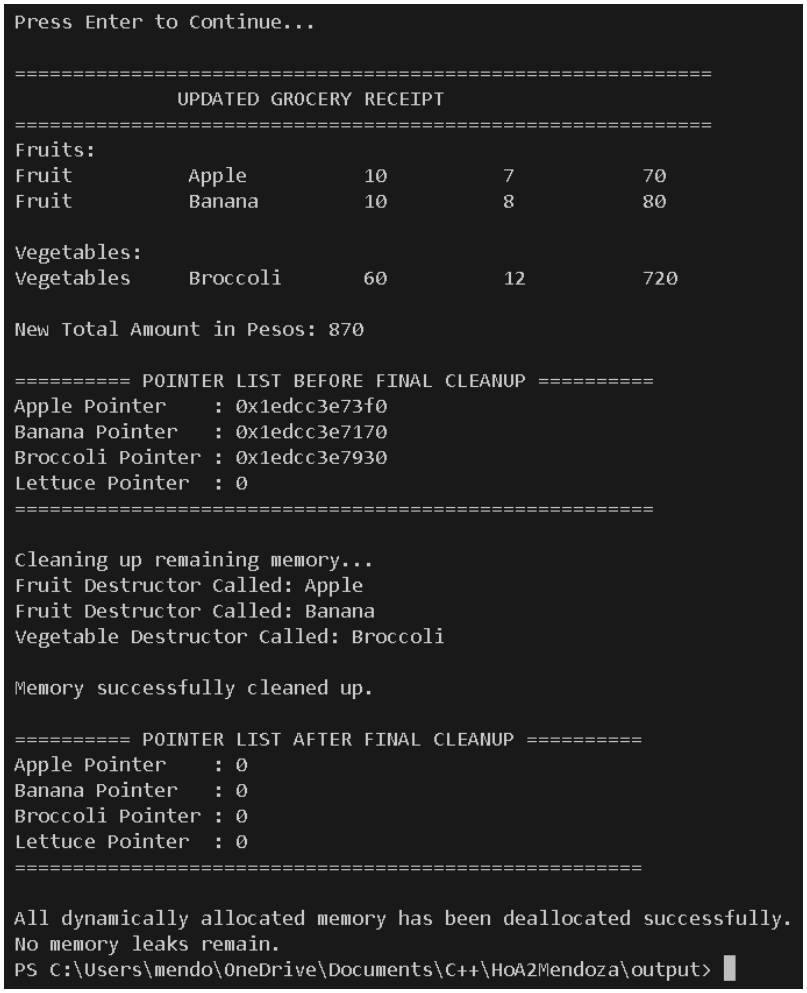
Deleting Lettuce...
Vegetable Destructor Called: Lettuce

Memory successfully deallocated.

========== POINTER LIST AFTER DELETION ==========
Apple Pointer    : 0x1edcc3e73f0
Banana Pointer   : 0x1edcc3e7170
Broccoli Pointer : 0x1edcc3e7930
Lettuce Pointer  : 0
================================================

Lettuce successfully deleted.
Lettuce pointer is now nullptr.

Press Enter to Continue...
```



Observation	The output displays the original grocery receipt and the total amount before deletion. It then shows the memory addresses of the grocery objects before and after Lettuce is deleted. The destructor message confirms that the Lettuce object was destroyed, while its pointer becomes nullptr. The updated receipt excludes Lettuce and shows the new total. Finally, the remaining objects are deleted, their destructor messages are displayed, and the pointer list shows nullptr, confirming that the dynamically allocated memory has been properly cleaned up.

Table 3. Sections and Answers to Supplementary Activity

CONCLUSION
Summary of Lessons Learned
I learned how objects can be created, accessed, copied, and deleted during program execution.
Analysis of Procedure
The procedure demonstrated how object-oriented programming concepts are implemented in C++. Objects were created using constructors, while copy constructors and copy assignment operators demonstrated how object data can be copied or assigned.
Analysis of Supplementary
The supplementary activity demonstrated how pointers store the memory addresses of dynamically allocated objects and how new, delete, and nullptr are used to manage memory. It also showed how properly deleting objects and setting their pointers to nullptr helps prevent memory leaks and makes memory management easier to visualize.
Concluding Statement / Feedback
This activity helped me understand pointers and dynamic memory allocation and ILO C helped me to keep track of the address more clearly because I was able to see how the pointers changed before and after deletion. The memory diagrams and pointer addresses made the process easier to visualize.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.